

UNIDAD 05

Combinaciones y Operaciones sobre Conjuntos

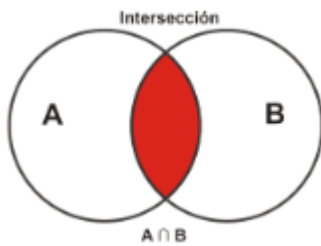
La capacidad de combinar información de múltiples tablas (Relaciones) es la característica definitoria de los sistemas de bases de datos relacionales. SQL proporciona dos mecanismos principales para lograr esto: las **operaciones de reunión (JOINS)**, que combinan columnas de dos tablas basadas en un criterio común, y las **operaciones de conjunto**, que combinan filas de dos consultas bajo una estructura de esquema compatible.

1. Reunión de Relaciones (JOINS)

El concepto de **JOIN** permite recuperar datos de dos o más tablas de manera simultánea. La clave para una reunión correcta es la cláusula **ON**, que especifica la condición lógica por la cual las filas de las tablas se emparejan, típicamente utilizando las **claves primarias (PK)** y **claves foráneas (FK)**.

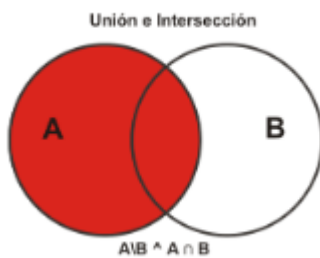
1.1. Tipos de JOINS y Cláusula ON

Tipo de JOIN	Función	Ejemplo en SQL Server
INNER JOIN	Devuelve solo las filas que tienen valores coincidentes en ambas tablas según la condición ON . Es la reunión más restrictiva.	SELECT ... FROM TablaA INNER JOIN TablaB ON TablaA.FK = TablaB.PK;
LEFT OUTER JOIN	Devuelve todas las filas de la tabla izquierda (TablaA) y las filas coincidentes de la tabla derecha (TablaB). Si no hay coincidencia, las columnas de la derecha muestran NULL .	SELECT ... FROM TablaA LEFT JOIN TablaB ON TablaA.FK = TablaB.PK;
RIGHT OUTER JOIN	Devuelve todas las filas de la tabla derecha (TablaB) y las filas coincidentes de la tabla izquierda (TablaA). Si no hay coincidencia, las columnas de la izquierda muestran NULL .	SELECT ... FROM TablaA RIGHT JOIN TablaB ON TablaA.FK = TablaB.PK;
FULL OUTER JOIN	Devuelve todas las filas cuando hay una coincidencia en cualquiera de las tablas, llenando con NULL donde no hay coincidencia en la tabla opuesta.	SELECT ... FROM TablaA FULL JOIN TablaB ON TablaA.FK = TablaB.PK;

Ejemplo de INNER JOIN en SQL Server (T-SQL):

Supongamos que tenemos **Cientes** (ID, Nombre) y **Pedidos** (ID_Pedido, ID_Cliente_FK, Monto). Queremos ver solo los clientes que han realizado pedidos.

```
SELECT
    C.Nombre,
    P.ID_Pedido,
    P.Monto
FROM Cientes C
INNER JOIN Pedidos P ON C.ID = P.ID_Cliente_FK;
-- Retorna todos los clientes (C) con pedidos (P)
```

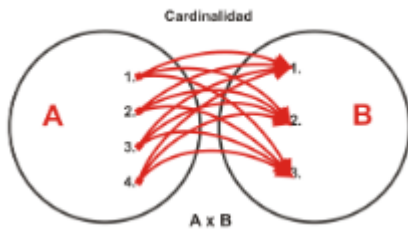
Ejemplo de LEFT JOIN en SQL Server (T-SQL):

Supongamos que tenemos **Cientes** (ID, Nombre) y **Pedidos** (ID_Pedido, ID_Cliente_FK, Monto). Queremos ver todos los clientes, incluso aquellos que no han realizado pedidos.

SQL

```
SELECT
    C.Nombre,
    P.ID_Pedido,
    P.Monto
FROM
    Cientes C
    LEFT JOIN Pedidos P ON C.ID = P.ID_Cliente_FK;
-- Retorna todos los clientes @
```

1.2. Producto Cartesiano (CROSS JOIN)



El **Producto Cartesiano** (también conocido como **CROSS JOIN**) es el resultado de combinar cada fila de la primera tabla con cada fila de la segunda tabla. Si la **TablaA** tiene N filas y la **TablaB** tiene M filas, el resultado tendrá $N \times M$ filas.

Este resultado generalmente es indeseado en consultas operacionales ya que no aplica ninguna condición de emparejamiento.

Ejemplo en SQL Server:

```
SELECT A.col1, B.col2
FROM TablaA A
CROSS JOIN TablaB B;
-- O simplemente: FROM TablaA A, TablaB B (sintaxis antigua y no deseada)
```

1.3. NATURAL JOIN (Nota en SQL Server)

El concepto de **NATURAL JOIN** (Estándar ANSI) empareja automáticamente las filas basándose en **todas las columnas que tienen el mismo nombre** en ambas tablas.

SQL Server (T-SQL) NO soporta explícitamente la cláusula NATURAL JOIN. Para lograr la misma funcionalidad, es necesario utilizar un **INNER JOIN** tradicional y especificar manualmente las condiciones de reunión para todas las columnas coincidentes.

2. Operaciones sobre Conjuntos

Las operaciones de conjunto permiten combinar los conjuntos de resultados **verticalmente** (filas sobre filas), a diferencia de los JOINS que combinan **horizontalmente** (columnas al lado de columnas).

Requisitos de compatibilidad de esquemas

Para que las operaciones de conjunto sean válidas, las consultas que se combinan deben cumplir con el requisito de **compatibilidad de esquemas**:

- 1. Deben tener el **mismo número de columnas**.
- 2. Las columnas correspondientes (por orden) deben tener **tipos de datos compatibles** (o que puedan ser convertidos implícitamente).

2.1. UNION y UNION ALL

Operador	Función Lógica	Eficiencia y Duplicados
UNION	Devuelve la unión lógica de dos conjuntos de resultados.	Realiza una ordenación y deduplicación interna; es más lento que UNION ALL.
UNION ALL	Devuelve la suma de todas las filas de los conjuntos.	No realiza deduplicación; es más eficiente y rápido, ideal cuando se sabe que los datos no se solapan (Ej: datos de 2023 vs. 2024).

Ejemplo en SQL Server:

```
-- Queremos una lista unificada de todos los nombres (empleados y departamentos)
SELECT Nombre_E AS Nombre, 'Empleado' AS Tipo FROM Empleados
UNION ALL -- Uso recomendado para evitar la sobrecarga de deduplicación
SELECT Nombre_D AS Nombre, 'Departamento' AS Tipo FROM Departamentos
ORDER BY Nombre;
```

2.2. INTERSECT y EXCEPT (Diferencia de conjuntos)

Operador	Función Lógica	Ejemplo
INTERSECT	Devuelve solo las filas que existen exactamente en ambos conjuntos de resultados.	Útil para encontrar usuarios que son tanto clientes como proveedores.
EXCEPT	Devuelve las filas que están en el primer conjunto de resultados (<i>ConsultaA</i>) pero no en el segundo (<i>ConsultaB</i>).	Útil para encontrar clientes que no han realizado una compra en el último mes.

Ejemplo de EXCEPT en SQL Server:

```
-- Encontrar IDs de clientes que están activos pero no han realizado un pedido en el año
SELECT ID_Cliente FROM ClientesActivos -- Primer conjunto
EXCEPT
SELECT ID_Cliente_FK FROM Pedidos_2024; -- Filas a excluir
```

(Nota: EXCEPT en SQL Server es equivalente a MINUS en Oracle/PostgreSQL).